



**QUEEN'S
UNIVERSITY
BELFAST**

A lightweight and hardware-efficient NTT FPGA accelerator for FHE applications

Zaman, M. A., Khan, M. B., Ahmad, W., Khalid, A., & O'Neill, M. (2025). A lightweight and hardware-efficient NTT FPGA accelerator for FHE applications. *IEEE Transactions on Circuits and Systems II: Express Briefs*. Advance online publication. <https://doi.org/10.1109/TCSII.2025.3633181>

Published in:
IEEE Transactions on Circuits and Systems II: Express Briefs

Document Version:
Peer reviewed version

Queen's University Belfast - Research Portal:
[Link to publication record in Queen's University Belfast Research Portal](#)

Publisher rights

Copyright 2025 the authors.

This is an accepted manuscript distributed under a Creative Commons Attribution License (<https://creativecommons.org/licenses/by/4.0/>), which permits unrestricted use, distribution and reproduction in any medium, provided the author and source are cited.

General rights

Copyright for the publications made accessible via the Queen's University Belfast Research Portal is retained by the author(s) and / or other copyright owners and it is a condition of accessing these publications that users recognise and abide by the legal requirements associated with these rights.

Take down policy

The Research Portal is Queen's institutional repository that provides access to Queen's research output. Every effort has been made to ensure that content in the Research Portal does not infringe any person's rights, or applicable UK laws. If you discover content in the Research Portal that you believe breaches copyright or violates any law, please contact openaccess@qub.ac.uk.

Open Access

This research has been made openly available by Queen's academics and its Open Research team. We would love to hear how access to this research benefits you. – Share your feedback with us: <http://go.qub.ac.uk/oa-feedback>

A Lightweight and Hardware-Efficient NTT FPGA Accelerator for FHE Applications

Moonis Amir Zaman, Muhammad Bilal Khan, Waqar Ahmad, Ayesha Khalid, Máire O'Neill

Abstract—The Number Theoretic Transform (NTT) and its inverse (INTT) are pivotal operations in Fully Homomorphic Encryption (FHE) schemes to facilitate efficient polynomial multiplication over modular rings. We propose a compact and hardware-efficient NTT/INTT architecture for FHE systems with a novel twiddle factor generator (TFG) for Radix-2 multi-path delay commutator (MDC) designs, saving up to 97.2% of on-chip memory by storing the twiddle factors (TF) for the initial 11 stages (for $\log_2(N) = 15$ to 17) and only the TF bases for on-the-fly generation in the remaining stages. DSP-efficient multipliers via non-standard tiling are designed to reduce DSP utilization as compared to standard tiling, without sacrificing performance. Benchmarking on comparable Xilinx FPGAs reveals that our design is the most compact, along with a significant efficiency advantage with up to $2.72\times$ reduction in average area-time product (ATP) and up to $2.57\times$ increase in throughput-per-equivalent-LUT (TPE) compared to state-of-the-art NTT designs. In various configurations, the architecture maintains lower BRAM and DSP utilization, and better overall efficiency, making it a scalable and efficient solution for real-world FHE deployments.

Index Terms—Fully Homomorphic Encryption (FHE), Number Theoretic Transform (NTT), Twiddle Factor Generation

I. INTRODUCTION

Polynomial arithmetic, in particular polynomial multiplication, is crucial to modern cryptographic schemes such as Fully Homomorphic Encryption (FHE) and Post-Quantum Cryptography (PQC) [1]. Polynomial multiplication tends to be a bottleneck operation in these schemes. The Number Theoretic Transform (NTT) comes into play here, reducing the complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log_2 n)$, making it a must-have component of FHE hardware designs.

There are two commonly used pipelined architectures for NTT, namely the single-path delay feedback (SDF) and multi-path delay commutator (MDC) architectures, with their respective performance-resource trade-offs. Different cryptographic schemes use a range of polynomial degrees (N) and modulus lengths, e.g., FHE generally uses N values from 2^{15} to 2^{17} with bit widths ($\log_2(q)$) of 50 to 64 bits. The reported work either optimizes NTT for specific parameter sets $[N, \log_2(q)]$, limiting its usability, or are run-time configurable for different parameters [2], resulting in large resource overhead. In contrast, compile-time configurable designs (parameters provided

before synthesis) such as [3], result in the most efficient architectures. While there is limited work in the literature for architectures supporting high-degree polynomial parameters, essential for next-generation cryptographic primitives like FHE, most of the reported work has large hardware overheads and high usage of memory due to twiddle factors and DSPs used in the standard integer multipliers implemented on FPGAs. Most of these reported designs precalculate and store the twiddle factors (TF) in memory [3]–[5] and majority do not support high-degree polynomials. For higher-order polynomials, this precalculation can contribute to a significant memory overhead.

In this paper, we present a compact and hardware-efficient NTT hardware architecture that can accommodate high-order polynomials from 2^{15} to 2^{17} . Our design can be configured at compile-time for different parameter sets. Our contributions are two-fold: First, to alleviate the high BRAM usage associated with storing precomputed twiddle factors (TF), we employ a hybrid approach of using some precomputed TF bases in memory and computing others on the fly. To reach an optimal split in this context, we carry out an exhaustive design space exploration to reach the optimal number of precomputed/on-the-fly stages to reach the most area-time product (ATP) efficient split, which we believe to be the first reported optimal twiddle factor generator (TFG) design for Radix-2 MDC architectures since most of the reported TFG designs for NTT are iterative architectures [2], [6], [7]. Second, since the generation of TFs requires additional multipliers, we also propose non-standard DSP tiling multiplier designs for various bit widths, that help reduce DSP usage compared to standard tiling based DSP multiplier designs used commonly in the literature. To our knowledge, we are the first to implement these DSP based optimizations on NTT architectures. Our experimental results show that we achieve $91.7\% \sim 97.2\%$ reduction in on-chip memory for TFs, $1.15\times \sim 2.72\times$ reduction in average area-time product (ATP) and $1.25\times \sim 2.57\times$ increase in throughput-per-equivalent-LUT (TPE) compared to the state-of-the-art NTT designs.

II. BACKGROUND

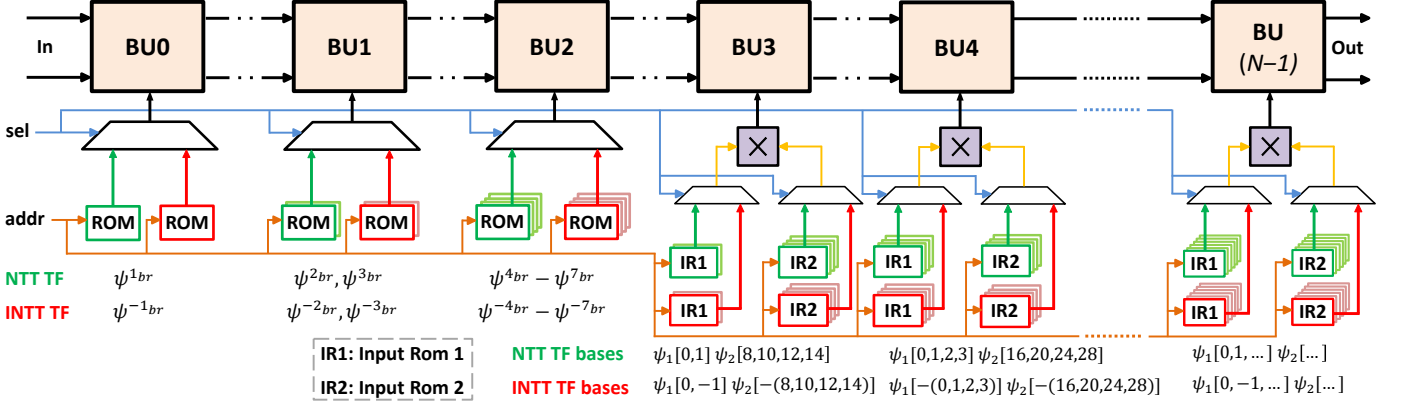
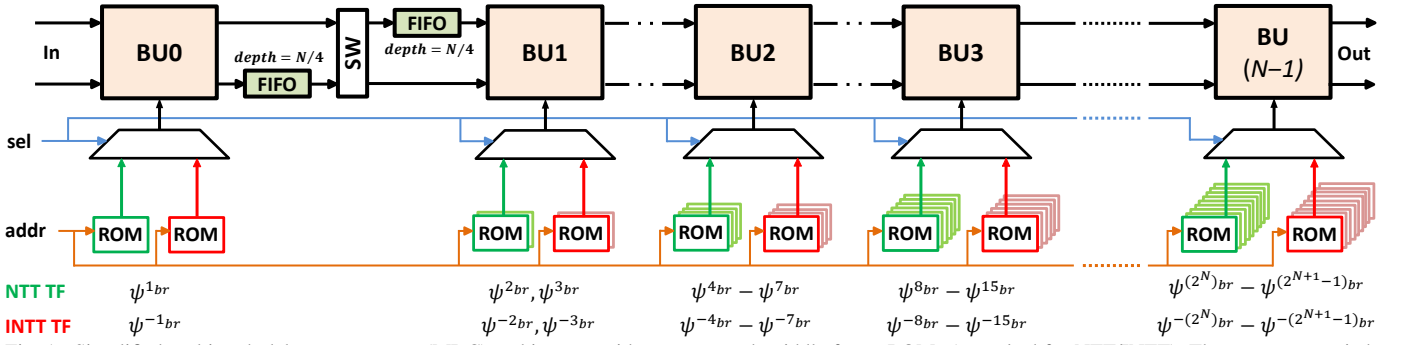
Number Theoretic Transform (NTT): The NTT is a fundamental operation in FHE schemes used to speed up polynomial multiplication. It is similar to the Fast Fourier Transform (FFT) except that it works over $\mathbb{Z}_q$ instead of $\mathbb{R}_q$, where $q \equiv 1 \pmod{N}$. An N -pt NTT takes a degree- N polynomial $a \in \mathbb{R}_q$ and outputs an N degree evaluation polynomial $\hat{a} \in \mathbb{R}_q$. In the NTT domain, polynomial multiplication is performed as point-wise multiplication, which is

This work is funded by the EPSRC-UK grant IQN (Integrated Quantum Networks) hub (EP/Z533208/1).

Moonis Amir Zaman and Muhammad Bilal Khan are with the National University of Sciences & Technology, Pakistan. E-mail: mza-man.bee21seecs@seecs.edu.pk, mkhan.bee21seecs@seecs.edu.pk.

Waqar Ahmad is with the Ghulam Ishaq Khan Institute of Engineering Sciences and Technology, Topi, Pakistan. E-mail: waqar.ahmad@giki.edu.pk.

Ayesha Khalid, and Máire O'Neill are with the Queen's University Belfast, UK. E-mail: a.khalid@qub.ac.uk, m.oneill@ecit.qub.ac.uk



much faster than the schoolbook polynomial multiplication. Most cryptographic applications work over the cyclotomic polynomial ring $\mathbb{R}_q = \mathbb{Z}_q[x]/(x^N + 1)$, which is called *negative-wrapped convolution (NWC)*. In NWC, the twiddle factors are primitive $2N$ -th roots of unity, ψ . The NTT and INTT are expressed as [8]:

$$\text{NTT}^\psi(a) = \hat{a}_j = \sum_{i=0}^{N-1} \psi^{2ij+i} a_i \mod q \quad (1)$$

$$\text{INTT}^{\psi^{-1}}(\hat{a}) = a_i = N^{-1} \sum_{j=0}^{N-1} \psi^{-(2ij+j)} \hat{a}_j \mod q \quad (2)$$

Two well-known approaches, Cooley-Tukey (CT) and Gentleman-Sande (GS), are typically used to implement these equations [8]. Both algorithms have the trade-off of bit reversed outputs, which require costly bit reversal operations to correct the data ordering, however by using CT for NTT and GS for INTT, we can avoid the need for reordering.

Twiddle Factor Generation: Twiddle factors (TF) are constants (ψ) used in the calculation of NTT/INTT. The higher the order of N , larger the number of TFs needed, e.g., for a unified NTT design with $N = 2^{17}$ with $\log_2(q) = 64$ bits, 262144 TFs are needed since $(2^{18} \times 64)/8 = (262144 \times 64)/8 \approx 2.10$ MB of data. To reduce on-chip memory, these TFs can be generated on-the-fly by executing the following recursive equation with a known initial value of ψ , [9]:

$$\psi^i = \psi^{i-1} \times \psi \mod q. \quad (3)$$

Since most NTT butterfly dataflows require TFs in bit-reversed order, additional TF bases need to be stored as well,

complicating the hardware design. Additionally, the generation of the next TF requires feedback from the previous output of the modular multiplier, which can result in additional latency for pipelined architectures, as modular multipliers typically take more than one cycle to execute. Reported work, including [3], [5], use precomputed TFs in memory, resulting in high memory overhead for large N . Other reported work, including [2], [6], [7], implement TFG modules but are limited to iterative NTT architectures. To the best of our knowledge, this work is the first reported optimal TFG for MDC architectures.

MDC architectures: Also called pipelined architectures, use only one butterfly unit (BU) for each stage of NTT/INTT, while iterative designs can have multiple BUs in parallel per stage, as shown in Fig. 1. These architectures take two input polynomial coefficients and generate two coefficients every cycle once their pipeline is full. Two FIFO (memory) units and a commutator switch (SW) are used at each stage to reorder the intermediate coefficients according to the required offset pattern for the next stage. The depth of the FIFO units depends on the formula [3]: $d = \frac{N}{2^{s+2}}$, where $s = 0, 1, \dots, \log_2(N) - 1$.

III. PROPOSED OPTIMIZATIONS FOR NTT/INTT

A. Twiddle Factor Generator (TFG)

The first stage (BU0) requires only two TFs, one each for NTT and INTT, while for subsequent stages the required TFs increase in powers of two. To reduce the on-chip memory for storing these TFs, we can generate most of them on-the-fly by storing a few TF bases and employing modular multipliers (MM). We aim to design a generator that does not require any feedback from the MM output to generate the next TF;

Algorithm 1 NTT Twiddle Factor Bases Precomputation for On-the-Fly Stages

Require: LOGN: $\log_2$ of transform size N , STORED: Initial stages stored, ψ_{table} : Table of N twiddle factors ($\psi^i \cdot R_w \pmod{q}$), R_w : Montgomery factor
Ensure: ROM_G1[0.. $L-S-1$][:], ROM_G2[0.. $L-S-1$][:] { $S = \text{STORED}, L = \text{LOGN}$ }

```

1: for  $s = \text{STORED}$  to  $\text{LOGN} - 1$  do
2:    $(f_1, f_2) \leftarrow \text{MFS}(2^s)$  {Factor  $2^s$  to minimize  $f_1 + f_2$ }
3:    $\text{addr\_bits1} \leftarrow \lceil \log_2(f_1) \rceil$ 
4:    $\text{addr\_bits2} \leftarrow \lceil \log_2(f_2) \rceil$ 
5:    $\text{rom\_idx} \leftarrow s - \text{STORED}$ 
6:   {Generate Input 1 bases (ROM_G1)}
7:   for  $i = 0$  to  $f_1 - 1$  do
8:      $\text{addr1} \leftarrow \text{BitRev}(i, \text{addr\_bits1})$ 
9:      $\text{idx} \leftarrow \text{BitRev}(i, \text{LOGN})$ 
10:    ROM_G1[rom_idx][addr1]  $\leftarrow \psi_{\text{table}}[\text{idx}]$ 
11:   end for
12:   {Generate Input 2 bases (ROM_G2)}
13:    $\text{bits\_inp2} \leftarrow \log_2(\text{BitRev}(i+1, \text{LOGN})) + 1$ 
14:   for  $j = 0$  to  $f_2 - 1$  do
15:      $\text{addr2} \leftarrow \text{BitRev}(j, \text{addr\_bits2})$ 
16:      $\text{rev\_j} \leftarrow \text{BitRev}(j, \text{bits\_inp2})$ 
17:      $\text{offset} \leftarrow 2^{\text{LOGN}-1-s}$ 
18:      $\text{idx} \leftarrow \text{rev\_j} + \text{offset}$ 
19:     ROM_G2[rom_idx][addr2]  $\leftarrow \psi_{\text{table}}[\text{idx}]$ 
20:   end for
21: end for

```

e.g., $256 \times 256 = 65536$, hence, to generate 65536 values we only require 512 unique factors. To determine the minimum number of TF bases for each stage, we wrote a Python script (Algorithm 1) to generate the Verilog for the TF base ROMs for each on-the-fly generated stage.

The algorithm takes as input the number of initial stages to be stored in memory, STORED, and the total number of stages, LOGN. It also takes a precomputed table of N twiddle factors, ψ_{table} , in which each TF is multiplied by the Montgomery factor, R_w , so that no correction operation is required later in the hardware. The output consists of two ROM arrays, ROM_G1 and ROM_G2, which store the bases for the two inputs to the modular multipliers for each stage from STORED to LOGN - 1. The algorithm iterates over each stage, factors 2^s into two components f_1 and f_2 to minimize their sum (MFS), and calculates the required address bits for each ROM. It then populates the ROMs with the appropriate TFs in bit-reversed index order (BitRev), ensuring that the generated TFs are correctly aligned for use in the NTT/INTT computations.

Now, the challenge is how to efficiently deploy the generator for MDC architectures. Firstly, it is obvious that placing a MM at each stage is impractical, as some of the initial stages require only a few distinct TFs. For $\log_2(q) = 64$, each MM uses 19 DSPs (with our multiplier designs) and around 1000 LUTs, which may increase the design area if implemented at every stage. Fig. 2 shows the MDC architecture with the TFG, where the first three stages store all TFs in ROM.

To determine the optimal split of stages with precomputed TFs and on-the-fly TFG, we carried out a design space exploration. We computed ATP for different extents of TFG for $N = 2^{15}$ and $N = 2^{16}$, with $\log_2(q) = 64$ in each case. We did not include results for the first three stages, as they are trivial and do not require a TFG. For the later stages, we

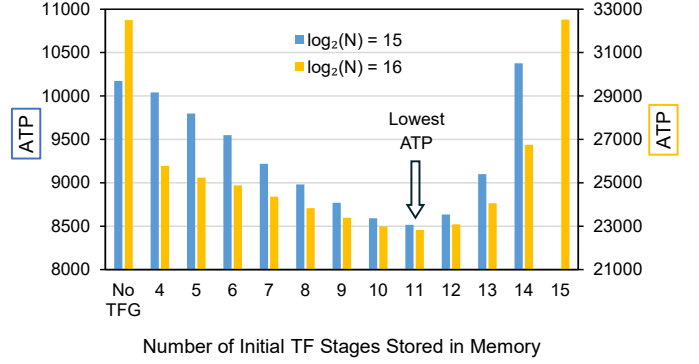


Fig. 3. ATP of the architecture for $\log_2(N) = 15/16$, for different TF stages, $\log_2(q) = 64$ bits. 'No TFG' refers to [3] with all TFs precomputed.

calculated the hardware efficiency as the area-time product (ATP) for each configuration, using the formula in [6], [10]:

$$\text{ATP} = (\text{LUT} + 100 \cdot \text{DSP} + 300 \cdot \text{BRAM}) \cdot \text{Lat.} \quad (4)$$

Where, Lat. or latency (execution time) in milliseconds is:

$$\text{Latency (ms)} = (\text{Execution CCs} \times 10^{-3}) / \text{Frequency (MHz)}$$

Fig. 3 shows the results for $N = 2^{15}$ and $N = 2^{16}$, with operating frequencies 196MHz and 186MHz, respectively. The optimal configuration for the TFG is observed when the lowest ATP is achieved, i.e., first 11 stages have all TFs stored in memory, while only a few TF bases are stored for the remaining stages for on-the-fly generation. We achieve a reduction of 19% and 42% in ATP, for $N = 2^{15}$ and $N = 2^{16}$, respectively. Note that our TFG provides much greater improvements for $N = 2^{16}$, as the reduction in BRAMs is more significant than in $N = 2^{15}$. The ATP formula we used weights BRAMs more heavily than LUTs and DSPs, hence the greater improvement.

A unified NTT/INTT stores $2N - 2$ TFs if all are precomputed, i.e., 65534 TFs for $N = 2^{15}$. In contrast, storing the first 11 TF stages requires only 5438 TFs for $N = 2^{15}$, with 4094 TFs for the first 11 stages ($2 \times 2^{11} - 2$) and 1344 TF bases for the later stages. The latter are calculated as: $2^{11} = 32 \times 64$, $2^{12} = 64 \times 64$, $2^{13} = 128 \times 64$, $2^{14} = 128 \times 128$, yielding $2 \times (32 + 64 + 64 + 64 + 128 + 64 + 128 + 128) = 1344$. This is a 91.7% reduction in on-chip memory as compared to traditional ROM-based storage. For $N = 2^{16}$, 6206 TFs are stored instead of 131070, with 4094 for the first 11 stages and 2112 TF bases, which is a 95.2% reduction in on-chip memory. This reduction increases to 97.2% for $N = 2^{17}$. The proposed TFG works in parallel to BUs and thus avoids contributing to the MDC architecture's latency. It ensures that the required TFs are available when needed by the BUs, thus introducing no stall cycles and maintaining a fully occupied pipeline.

B. DSP-Efficient Multipliers

The integer multiplier is a crucial component of NTT/INTT. Furthermore, the on-the-fly generation of TFs necessitates additional multipliers, leading to increased DSP utilization. A common technique for integer multiplication is the standard tiling (ST) approach [11], employed also by [3], that partitions the input operands into smaller chunks. The DSP48E1 multiplier in 7-series FPGAs is a 25×18 bits signed multiplier. To emulate unsigned multiplication, the MSB of each operand

is set to 0, effectively performing 24×17 unsigned multiplication. Similarly, the DSP48E2 in Ultrascale+ FPGAs supports 27×18 signed and 26×17 unsigned multiplication. This approach of using unsigned numbers avoids any logic required for signed to unsigned number conversions, that can potentially reduce the maximum frequency of the overall design.

Fig. 4(a) and (e) show the standard tiling configurations for DSP48E1 and DSP48E2, respectively. In Fig. 4(a), for a 52×52 bit multiplication, the first three DSPs of the last column perform 4×17 bit multiplications each and the final DSP performs a 4×1 bit multiplication, which is not an efficient use of those DSPs. Similar issues can be observed for other bit sizes such as 36, 54, and 64 (mainstream bit widths of FHE accelerators), as well as in Fig. 4(e). Hence, we propose more efficient DSP-mapping techniques for various bit ranges.

Our mapping algorithm dynamically partitions input operands ($a \times b$) based on the bit width of the modulus ($\log_2(q)$) and the DSP dimensions available in the target FPGA (24×17 or 26×17). It determines the optimal DSP count by analyzing operand coverage in both width and height, ensuring full multiplication representation without redundancy. Operands are segmented into DSP-aligned chunks, and partial products from each chunk pair are aligned via left shifts and accumulated by a carry-save adder (CSA) tree to reduce carry propagation delay. A final two-operand addition produces the product. This structured approach minimizes DSP usage while preserving full precision and high operating frequency across varying operand sizes.

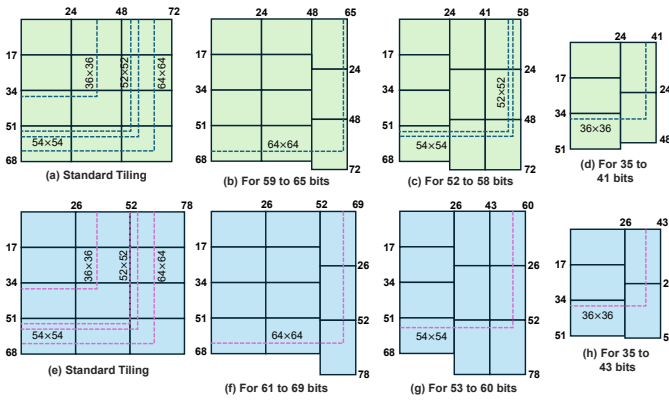


Fig. 4. Standard vs Non-standard tiling for different bit widths. (a) to (d) use DSP48E1, while (e) to (h) use DSP48E2.

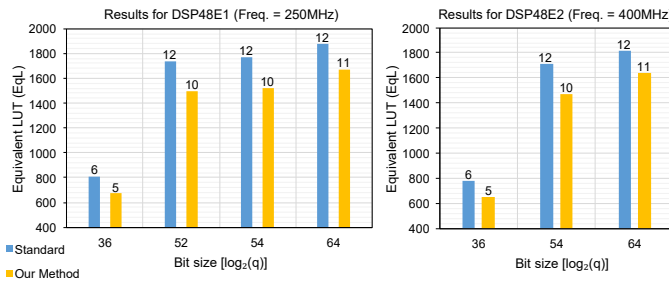


Fig. 5. Comparison of Equivalent LUT (EqL [6], [10]) used in standard tiling vs. our DSP mapping for DSP48E1 (left) and DSP48E2 (right). The numbers above each bar show the number of DSPs used. EqL = LUT + $100 \times$ DSP + $300 \times$ BRAM (BRAM usage is 0 in all cases).

For DSP48E1, Fig. 4(b) and Fig. 4(d) present our proposed design for input bit-widths of 59 to 65 bits and 35 to 41 bits, respectively. In both configurations, the last DSP in each

row is vertically aligned, with the remaining bits of input a mapped to the shorter side and multiplied by up to 24 bits of input b mapped to the longer side of the DSP. This results in a reduction of one DSP unit compared to standard tiling. A similar arrangement for DSP48E2 in Fig. 4(f) and Fig. 4(h) targets bit-widths of 61 to 69 bits and 35 to 43 bits, respectively, with b divided into 26-bit chunks.

For 52 to 58 bit inputs in DSP48E1, Fig. 4(c) divides input a into one 24-bit and two 17-bit chunks, and input b into 17-bit and 24-bit chunks. The 24-bit chunk of a multiplies with the 17-bit chunk of b , and the 17-bit chunk of a multiplies with the 24-bit chunk of b , reducing two DSPs compared to standard tiling. A similar design for DSP48E2 in Fig. 4(g) reduces two DSPs for bit-widths of 53 to 60 bits.

Fig. 5 presents a comparative analysis between standard tiling and our proposed methods, for different bit sizes, on both DSP48E1 and DSP48E2 architectures. We used a Virtex-7 FPGA for DSP48E1 and ZCU102 (Ultrascale+) to implement DSP48E2 designs. The results show up to 16.8% area reduction (EqL) due to more efficient DSP utilization in the proposed designs compared to standard methods.

IV. HARDWARE IMPLEMENTATION RESULTS

We implemented our proposed designs on various Xilinx FPGAs using Xilinx Vivado 2019.1. We configured our TFG to precomputed 11 stages, as discussed in Section III-A, and used a unified-NWC architecture. Table I shows the Post-Place-and-Route (Post-PAR) results comparison between different NTT designs in the literature for large polynomial degree sizes. We calculated the throughput (TP) as done in [2]. We calculated the throughput-per-equivalent-LUT (TPE), where the equivalent LUT (EqL) for area utilization is calculated as done in [6]. The area-time product (ATP, ref. Section III-A) to determine hardware efficiency is also computed. Note that we used the average of the performance metrics for comparison, as the NTT/INTT operations are used for many inputs consecutively, in real world FHE deployments.

For $N = 2^{15}$, $\log_2(q) = 64$, we compared our design to [3], which is a state-of-the-art pipelined NTT architecture, which can be configured as SDF or MDC. We compared our work to a unified MDC configuration of this work. Our design achieves $1.33 \times$ better average ATP and $1.35 \times$ equivalent LUT, along with $1.35 \times$ better average TPE. Moreover, we use $3.03 \times$ fewer BRAMs due to our TFG, at the expense of $1.20 \times$ more DSPs, whereas [3] stores all its TFs in memory and therefore requires fewer multipliers overall than our work.

For $N = 2^{16}$, $\log_2(q) = 52$, we compared our work to [3] and two iterative NTT architectures [6], [7]. Compared to [3], our work uses $1.18 \times$ more DSPs due to on-the-fly computation of TFs, however it uses $3.16 \times$ fewer BRAMs. Both [6], [7] generate TFs on the fly, however, our work utilizes the lowest number of DSPs and BRAMs. Since iterative architectures can be parallelized, they are faster as compared to pipelined architectures. While [6] and [7] take nearly $2 \times$ fewer average clock cycles, and $1.69 \times$ and $1.58 \times$ lower average latency, respectively, our design outperforms them in average ATP and TPE. Specifically, it achieves $1.56 \times$ and $1.49 \times$ better ATP, and $1.60 \times$ and $1.50 \times$ better TPE, respectively.

TABLE I
POST-PAR IMPLEMENTATION RESULTS OF OUR WORK VS. STATE-OF-THE-ART NTT/INTT DESIGNS FOR VARIOUS FHE PARAMETERS

[Ref.] / Year	N	$\log_2(q)$ (bits)	Freq. (MHz)	kLUT	kFF	DSP	BRAM	Avg. Lat. (cc)*	Avg. Lat. (μ s)	kEqL $^\alpha$	Avg. ATP	ATP impro.	Avg. TP (Mbps) $^\beta$	Avg. TPE $^\gamma$	TPE impro.	Device
Our work	2^{15}	64	212	43.6	33.2	361	63	16,547	78.1	98.6	7,701	<i>ref</i>	26,852	0.27	<i>ref</i>	Virtex-7
[3] / 2024 †	2^{15}	64	196	34.4	27.8	300	191	16,547	84.4	121.7	10,272	$1.33\times\uparrow$	24,847	0.20	$1.35\times\uparrow$	Virtex-7
Our work	2^{16}	52	250	38.0	26.7	378	96	33,096	132.4	104.6	13,860	<i>ref</i>	25,739	0.24	<i>ref</i>	ZCU102
[3] / 2024 †	2^{16}	52	243	30.5	22.3	320	304	33,096	136.1	153.7	20,918	$1.51\times\uparrow$	25,039	0.16	$1.50\times\uparrow$	ZCU102
[6] / 2024	2^{16}	52	210	157	117	896	98	16,445	78.4	276	21,653	$1.56\times\uparrow$	43,467	0.15	$1.60\times\uparrow$	ZCU102
[7] / 2022	2^{16}	52	200	149	91	564	137	16,776	83.9	246.5	20,681	$1.49\times\uparrow$	40,618	0.16	$1.50\times\uparrow$	ZCU102
Our work	2^{16}	64	196	47.3	36.3	399	127	33,096	168.8	125.3	21,159	<i>ref</i>	24,847	0.20	<i>ref</i>	Virtex-7
[3] / 2024 †	2^{16}	64	186	37.7	29.7	320	383	33,096	176	184.6	32,498	$1.53\times\uparrow$	23,831	0.13	$1.53\times\uparrow$	Virtex-7
[2] / 2024 ‡	2^{16}	60	250	274.4	83.1	630	241	16,552	66.2	409.7	27,122	$1.28\times\uparrow$	59,398	0.14	$1.42\times\uparrow$	U250
[12] / 2023	2^{16}	64	260	70	93	132	236	54,000	207	154	31,878	$1.50\times\uparrow$	20,262	0.13	$1.53\times\uparrow$	Virtex-7
[5] / 2023	2^{16}	60	250	74.5	61.4	288	883	16,628	66.5	368.2	24,485	$1.15\times\uparrow$	59,130	0.16	$1.25\times\uparrow$	VU37P
Our work	2^{17}	60	250	51.1	36.7	437	218	66,192	264.7	160.6	42,510	<i>ref</i>	29,710	0.18	<i>ref</i>	U250
[2] / 2024 ‡	2^{17}	60	250	274.4	83.1	630	241	32,997	131.9	409.7	54,039	$1.27\times\uparrow$	59,623	0.14	$1.28\times\uparrow$	U250
[13] / 2024	2^{17}	60	250	119.9	67.6	470	229	41,207	164.8	235.6	38,827	$1.09\times\downarrow$	47,720	0.20	$1.11\times\downarrow$	U250
[14] / 2025	2^{17}	62	280	118.3	119.2	840	822	72,167	257.7	448.9	115,688	$2.72\times\uparrow$	31,534	0.07	$2.57\times\uparrow$	VU13P

NOTE: The $\uparrow$ and $\downarrow$ indicate superior or inferior metrics compared to others. *: Avg. latency for 100 operations. †: Configured for MDC-Unified-NWC and Montgomery reduction.

‡: Run-time configurable for $N = 2^{16}/2^{17}$. α : EqL = LUT + 100×DSP + 300×BRAM [6] β : Avg. TP = Total number of bits / Avg. Latency (μ s) [2] γ : Avg. TPE = Avg. TP / EqL

For $N = 2^{16}$, $\log_2(q) = 60 - 64$, we compared our design to [2], [3], [5], [12]. Our architecture achieves the best average ATP from $1.15\times$ to $1.53\times$ better than the other designs, while also achieving $1.25\times$ to $1.53\times$ better average TPE. Furthermore, our design has the lowest BRAM usage. [2] uses Radix-2⁴ BUs which processes more coefficients in parallel as compared to our Radix-2 BUs, hence they have high throughput however, our TPE is still higher. [12] utilizes the fewest number of DSPs, however, it has the highest number of average clock cycles resulting in the longest latency. [5] achieves very low latency despite comparatively lower number of DSPs, however, it uses the highest number of BRAMs resulting in $1.15\times$ and $1.25\times$ worse average ATP and TPE, respectively, than our design.

Finally, for $N = 2^{17}$ and $\log_2(q) = 60 - 62$, we compared our work to [2], [13], [14]. [2] can be configured for $N = 2^{16}$ or $N = 2^{17}$ at run time; hence, their resource usage is identical for each parameter. Our work achieves a $1.27\times$ higher average ATP and $1.28\times$ higher average TPE. Additionally, our design is more compact as it uses $2.55\times$ fewer equivalent LUTs and $1.44\times$ fewer DSPs. [13] achieves better average ATP and TPE than our work, however, it uses $1.46\times$ more equivalent LUTs and $1.07\times$ more DSPs than our design. Compared to [14], we achieve significant improvements of $2.72\times$ and $2.57\times$ in average ATP and TPE, respectively, however, their work supports the Residue Number System based FHE schemes without need for reconfiguration when switching modulus.

V. CONCLUSION

This paper proposes novel optimization techniques for MDC NTT/INTT architectures for FHE applications. We achieve a reduction of up to 97.2% in memory utilization by employing a novel TFG. Combined with DSP-efficient multiplier designs, these optimizations deliver significantly better average ATP and TPE, as compared to most reported literature. Future work includes extending the TFG to support different radix configurations of pipelined architectures, such as Radix-4, and developing a generator for single-path delay feedback (SDF) NTT architectures.

REFERENCES

- [1] J. Xie, W. Zhao, H. Lee, D. B. Roy, and X. Zhang, "Hardware Circuits and Systems Design for Post-Quantum Cryptography—A Tutorial Brief," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 3, pp. 1670–1676, 2024.
- [2] R. Mareta, A. Satriawan, P. N. Duong, and H. Lee, "A Bootstrapping-Capable Configurable NTT Architecture for Fully Homomorphic Encryption," *IEEE Access*, vol. 12, pp. 52911–52921, 01 2024.
- [3] F. Hirner, A. C. Mert, and S. S. Roy, "Proteus: A Pipelined NTT Architecture Generator," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 32, no. 7, pp. 1228–1238, 2024.
- [4] Y. Geng, X. Hu, and Z. Wang, "GS-MDC: High-Speed and Area-Efficient Number Theoretic Transform Design," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 71, no. 12, pp. 4974–4978, 2024.
- [5] S. Kurniawan, P. Duong-Ngoc, and H. Lee, "Configurable Memory-Based NTT Architecture for Homomorphic Encryption," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, pp. 3942–3946, 06 2023.
- [6] F. Krieger, F. Hirner, A. C. Mert, and S. S. Roy, "OpenNTT: An Automated Toolchain for Compiling High-Performance NTT Accelerators in FHE," *Cryptology ePrint Archive*, Paper 2024/1740, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1740>
- [7] P. Duong-Ngoc, S. Kwon, D. Yoo, and H. Lee, "Area-Efficient Number Theoretic Transform Architecture for Homomorphic Encryption," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 3, pp. 1270–1283, 2023.
- [8] A. Satriawan, I. Syafalni, R. Mareta, I. Anshori, W. Shalannanda, and A. Barra, "Conceptual Review on Number Theoretic Transform and Comprehensive Review on Its Implementations," *IEEE Access*, vol. 11, pp. 70288–70316, 2023.
- [9] N. Im, H. Yang, Y. Eom, S.-C. Park, and H. Yoo, "Efficient Twiddle Factor Generators for NTT," *Electronics*, vol. 13, no. 16, 2024.
- [10] Z. Ye, R. C. C. Cheung, and K. Huang, "PipeNTT: A Pipelined Number Theoretic Transform Architecture," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 69, no. 10, pp. 4068–4072, 2022.
- [11] D. B. Roy, D. Mukhopadhyay, M. Izumi, and J. Takahashi, "Tile Before Multiplication: An Efficient Strategy to Optimize DSP Multiplier for Accelerating Prime Field ECC for NIST Curves," in *2014 51st ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2014, pp. 1–6.
- [12] Z. Li, J. Ren, G. Du, Z. Tu, X. Wang, Y. Yin, and Y. Ouyang, "An Area-Efficient Large Integer NTT-Multiplier Using Discrete Twiddle Factor Approach," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 2, pp. 751–755, 2023.
- [13] R. Mareta and H. Lee, "Compact 217 NTT Architecture for Fully Homomorphic Encryption," in *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2024, pp. 1–5.
- [14] Q. Dang Truong, P. Duong-Ngoc, and H. Lee, "Hybrid Number Theoretic Transform Architecture for Homomorphic Encryption," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 33, no. 7, pp. 2039–2043, 2025.